WA3396 Lab Guide

Intro to Web and HTML Fundamentals Liberty

ascendient learning
Part of Accenture

© 2026 Ascendient, LLC

Revision 2.0.0 published on 2026-03-13.

No part of this book may be reproduced or used in any form or by any electronic, mechanical, or other means, currently available or developed in the future, including photocopying, recording, digital scanning, or sharing, or in any information storage or retrieval system, without permission in writing from the publisher.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

To obtain authorization for any such activities (e.g., reprint rights, translation rights), to customize this book, or for other sales inquiries, please contact:

Ascendient, LLC
1 California Street Suite 2900
San Francisco, CA 94111
https://www.ascendientlearning.com/

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com
Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Lab Materials and Environment Guidelines

Ascendient Learning is pleased to provide lab materials and environments as part of your learning experience. To ensure a safe and effective learning experience for everyone:

✅ **You may:**

- Use the lab environment for your own coursework.
- Save any important files or data from the environment to your local computer or online storage (if allowed by your employer).
- Retain lab files for your personal use after the class ends.

❌ **You may not:**

- Share your lab materials or environment with anyone outside of your course.
- Store any personal, private, confidential, or proprietary company information within the lab environment.
- Rely on the lab environment for data storage, as all work is permanently deleted after each class.
- Misuse the lab environment for any illegal, malicious, or non-class-related activities.

> ⓘ `Important`
>
> The lab environment is under Ascendient Learning's control and is not a part of your organization's IT ecosystem. Do not place your organization's information into the lab environment and always follow your organization's policies and procedures for working with IT systems.

Table of Contents

1. Learn the Basics of HTML Using Visual Studio Code 6
1.1. Open Visual Studio Code 7
1.2. Create the HTML File 9
1.3. Save the HTML Page 10
1.4. View the HTML Page in Your Browser 10
1.5. Review 11
2. Create a Basic HTML5 Layout 12
2.1. Open the Existing Lab Folder 13
2.2. Create the Template File 13
2.3. Test the Template with NGINX 15
2.4. Review 15
3. Create a Registration Form 16
3.1. Open the Existing Lab Folder 17
3.2. Create the Registration Form File 17
3.3. Add Name Fields 18
3.4. Add a Drop-Down Menu 18
3.5. Add Radio Buttons 19
3.6. Add Checkboxes 19
3.7. Add a Phone Field and Address Area 20
3.8. Add a Password Field 20
3.9. Add the Submit Button 21
3.10. Create the Thank-You Page 22
3.11. Test with NGINX 23
3.12. Review 24
4. Improve Your Forms Using HTML5 25
4.1. Create the Employee Registration Form 26
4.2. Add the Form Structure 27
4.3. Reuse the Thank-You Page 30
4.4. Test with NGINX 30

4.5. Review 32
5. HTML Video and Audio 33
5.1. Example: Build a Simple Media Page 34

Learn the Basics of HTML Using Visual Studio Code

LAB 1

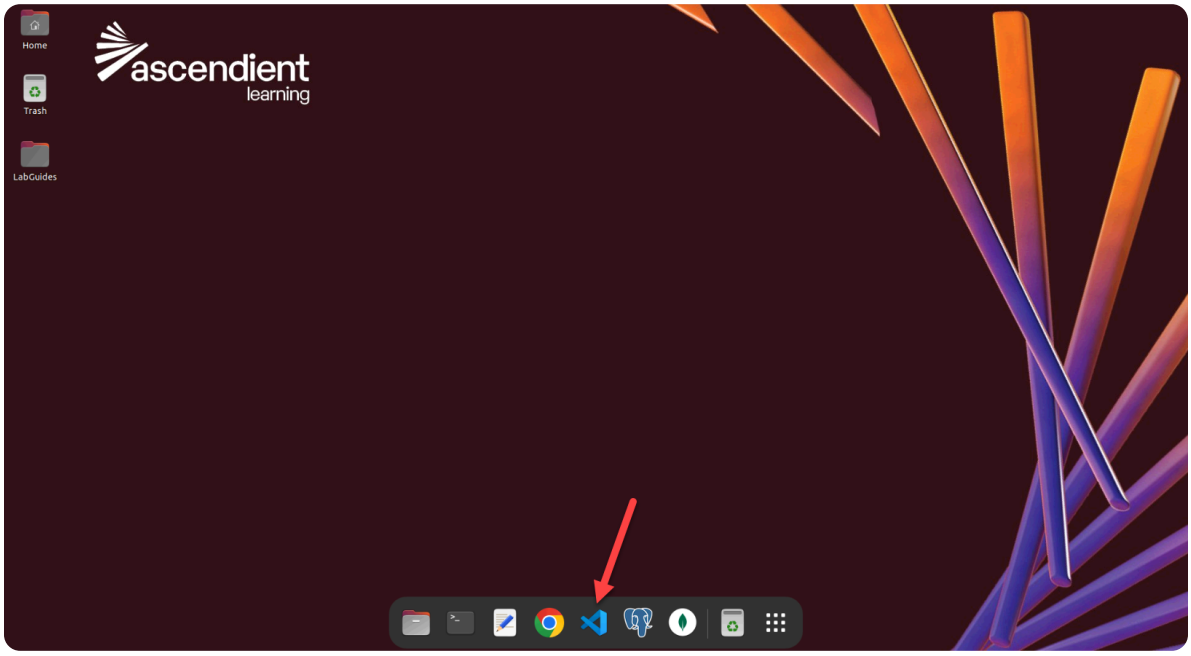
Web pages can be created and edited with many different tools.

For this first HTML exercise, you will use Visual Studio Code to create a simple web page.

Using a basic editor helps you focus on the HTML itself before moving on to more advanced tools and workflows.

1.1. Open Visual Studio Code

1. Start Visual Studio Code.

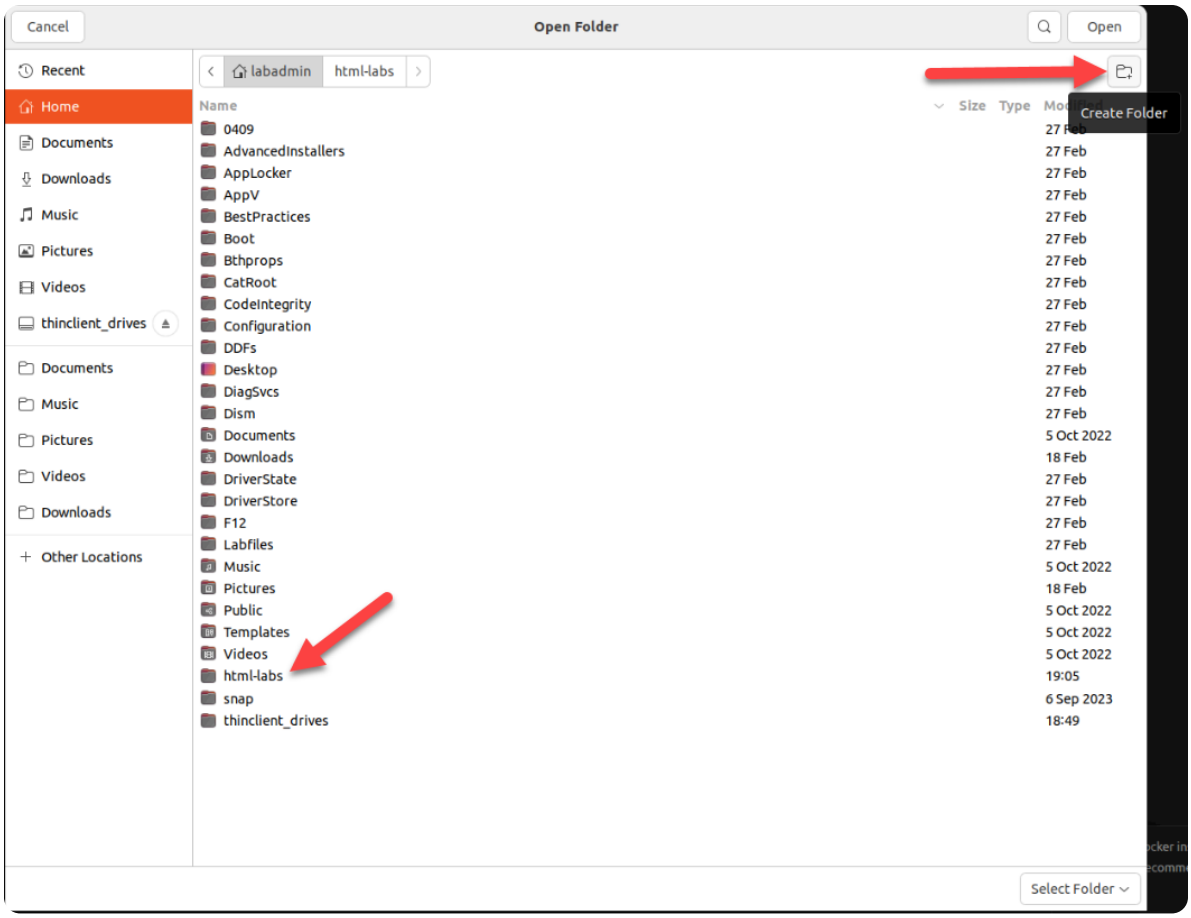


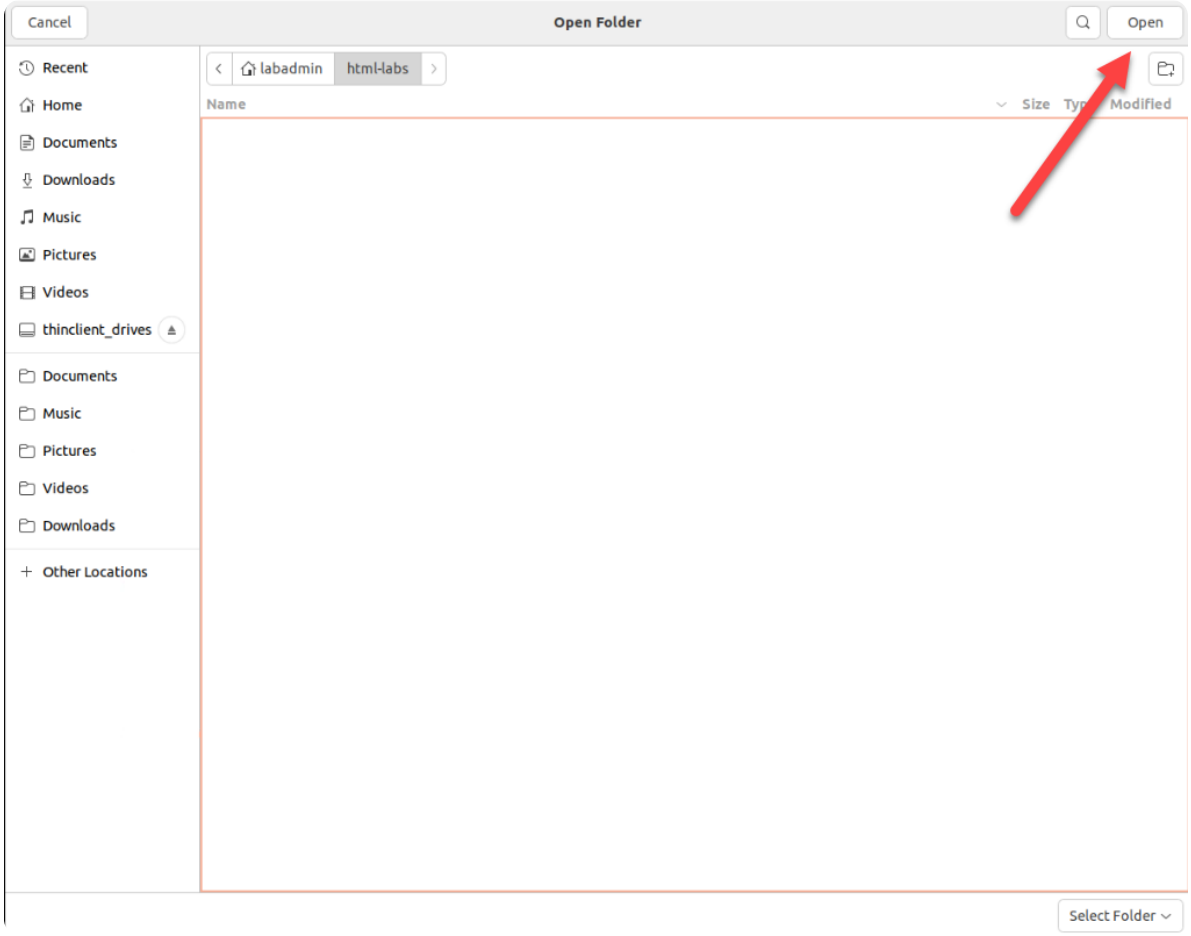
2. If asked to authenticate, enter **labadmin** to allow Visual Studio Code to make changes to your computer

3. If you see a welcome page, you can close it by clicking the **X** in the tab.

4. Select **File** > **Open Folder**.

5. Create a folder named **html-labs** in your **Home** folder, then open it in Visual Studio Code.


Cancel
Open Folder
Open
Recent
Home
Documents
Downloads
Music
Pictures
Videos
thinclient_drives
Documents
Music
Pictures
Videos
Downloads
Other Locations
labadmin
html-labs
Create Folder
Name
Size
Type
0409
AdvancedInstallers
AppLocker
AppV
BestPractices
Boot
Bthprops
CatRoot
CodeIntegrity
Configuration
DDFs
Desktop
DiagSvcs
Dism
Documents
Downloads
DriverState
DriverStore
F12
Labfiles
Music
Pictures
Public
Templates
Videos
html-labs
snap
thinclient_drives
27 Feb
27 Feb
27 Feb
27 Feb
27 Feb
27 Feb
27 Feb
27 Feb
27 Feb
27 Feb
27 Feb
27 Feb
27 Feb
27 Feb
27 Feb
5 Oct 2022
18 Feb
27 Feb
27 Feb
27 Feb
27 Feb
5 Oct 2022
18 Feb
5 Oct 2022
5 Oct 2022
5 Oct 2022
19:05
6 Sep 2023
18:49
Select Folder



6. Select **Yes, trust the authors** if prompted.

1.2. Create the HTML File

1. In the Explorer panel, click **New File**.
2. Name the file **index.html**.
3. Enter the following HTML code:

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>
```

```
<p>My first paragraph.</p>

</body>
</html>
```

1.3. Save the HTML Page

1. Save the file by selecting **File** > **Save** or by pressing **Ctrl+S**.
2. Confirm that the file name is **index.html**.

> Tip
>
> You can use either **.htm** or **.html** as the file extension.

1.4. View the HTML Page in Your Browser

1. Open the file manager and browse to the **html-labs** folder.
2. Double-click **index.html** to open it in your default web browser.

The page should display a heading that says **My First Heading** and a paragraph that says **My first paragraph.**

+ image::assets/preview-page-filesystem.png[Preview Page,height=460]

1. Switch back to Visual Studio Code.
2. Open a terminal by selecting **Terminal** > **New Terminal**.
3. Copy the file to the NGINX document root by running the following command:

```
sudo cp ~/html-labs/index.html /usr/share/nginx/html/index.html
```

4. If prompted for your password, enter **labadmin** to allow the file copy.

5. You do not need to restart NGINX after replacing a static HTML file.
6. Verify that the file was copied successfully by running the following command:

```
cat /usr/share/nginx/html/index.html
```

7. Open a browser and go to **http://localhost/**.
8. Confirm that the page displays the same heading and paragraph through NGINX.

1.5. Review

In this lab, you created a simple HTML document in Visual Studio Code, copied it to the NGINX document root, and viewed it in a web browser.

Create a Basic HTML5 Layout

LAB 2

This lab continues from the Learn the Basics of HTML Using Visual Studio Code lab.

If you have not completed that lab yet, do that first before continuing.

In this lab, you will build a reusable HTML5 template that you can use as the starting point for later exercises.

You will learn how to structure a simple HTML5 document and define the DOCTYPE, head, character encoding, title, and body elements.

2.1. Open the Existing Lab Folder

1. If not already open, start Visual Studio Code.
2. Select **File** > **Open Folder**.
3. Open the **html-labs** folder you created in the previous lab.
4. If prompted, select **Yes, trust the authors**.

2.2. Create the Template File

In this part, you will create an HTML5 template. The template will include the following elements:

- DOCTYPE
- html
- head
- meta
- title
- body

    1. In the Explorer panel, click **New File**.

2. Name the file **html5Template.html**.

3. Enter the DOCTYPE declaration, followed by the opening **html** element:

```
<!DOCTYPE html>
<html>
```

The DOCTYPE declaration must be the first line in an HTML5 document.

```
Note: The DOCTYPE declaration is case-insensitive. For example, both of the following are valid:

<!DOCTYPE HTML>

<!doctype html>
```

1. Add the **head** element with the document character encoding and title:

```
<head>
    <meta charset="utf-8"/>
    <title>HTML5 Template</title>
</head>
```

The title appears in the browser tab, and the meta charset declaration tells the browser to use UTF-8 encoding.

1. Add the **body** element and close the **html** element:

```
<body>
    Content goes here...
</body>

</html>
```

2. Your completed file should look like this:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8"/>
    <title>HTML5 Template</title>
</head>
<body>
    Content goes here...
</body>

</html>
```

3. Save the file by selecting **File** > **Save** or by pressing **Ctrl+S**.

2.3. Test the Template with NGINX

A web server stores and delivers web content to a browser. In this environment, NGINX is the web server.

1. Open a terminal in Visual Studio Code by selecting **Terminal** > **New Terminal**.
2. Copy the template file to the NGINX document root:

```
sudo cp ~/html-labs/html5Template.html /usr/share/nginx/html/html5Template.html
```

3. If prompted for your password, enter **labadmin**.
4. Verify that the file was copied successfully:

```
cat /usr/share/nginx/html/html5Template.html
```

5. Open a browser.
6. Browse to the following URL:

   http://localhost/html5Template.html

7. Confirm that the page displays the text **Content goes here...**.

localhost refers to the computer you are currently using. In this case, the browser is loading the file from the local NGINX web server.

2.4. Review

In this lab, you created a basic HTML5 template in Visual Studio Code, copied it to the NGINX document root, and viewed it in a web browser.

Create a Registration Form

LAB 3

This lab continues from the previous two labs:

- Learn the Basics of HTML Using Visual Studio Code
- Create a Basic HTML5 Layout

If you have not completed those labs yet, do that first before continuing.

In this lab, you will create a simple registration form and a thank-you page. You will use several common HTML form elements and test the finished files through NGINX.

3.1. Open the Existing Lab Folder

1. If not already open, start Visual Studio Code.
2. Select **File** > **Open Folder**.
3. Open the **html-labs** folder you created in the previous labs.
4. If prompted, select **Yes, trust the authors**.

3.2. Create the Registration Form File

1. In the Explorer panel, create a new file named **RegistrationForm.html**.
2. If you want to reuse the structure from the previous lab, you can use **html5Template.html** as a reference.
3. Enter the following starter code:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8"/>
    <title>Registration Form</title>
</head>
```

```
<body>
    <h1>Registration Form</h1>

</body>
</html>
```

4. Save the file.

3.3. Add Name Fields

Next, you will add a form element and two text fields for first and last name input.

1. Add the form element below the **h1** heading:

```
<form action="ThankYou.html">
</form>
```

2. Add a label and text box for the first name:

```
<label>Firstname:</label>
<input type="text" name="firstname" size="15"/> <br> <br>
```

3. Add a label and text box for the last name:

```
<label>Lastname:</label>
<input type="text" name="lastname" size="15"/> <br> <br>
```

4. Save the file.

3.4. Add a Drop-Down Menu

You will now add a drop-down list so the user can select a course.

1. Add the following code inside the form:

```
<label>Course:</label>
<select>
    <option value="Course">Course</option>
    <option value="BCA">BCA</option>
```

```
    <option value="BBA">BBA</option>
    <option value="B.Tech">B.Tech</option>
    <option value="MBA">MBA</option>
    <option value="MCA">MCA</option>
    <option value="M.Tech">M.Tech</option>
</select>
<br>
<br>
```

2. Save the file.

3.5. Add Radio Buttons

Radio buttons allow the user to choose one option from a group.

1. Add the following code for a gender selection:

```
<label>Gender:</label><br>
<input type="radio" name="gender"/> Male <br>
<input type="radio" name="gender"/> Female <br>
<input type="radio" name="gender"/> Other
<br>
<br>
```

2. Save the file.

3.6. Add Checkboxes

Checkboxes allow the user to select one or more options.

1. Add the following code for hobbies:

```
<label>Hobbies:</label>
<br>
<input type="checkbox" name="Programming"/> Programming <br>
<input type="checkbox" name="Cricket"/> Cricket <br>
<input type="checkbox" name="Football"/> Football <br>
<input type="checkbox" name="ReadingNovel"/> Reading Novel
```

```
<br>
<br>
```

2. Save the file.

3.7. Add a Phone Field and Address Area

Next, you will add a text field for a phone number and a text area for a mailing address.

1. Add the following code for a phone number field:

```
<label>Phone:</label>
<input type="text" name="phone" size="10"/>
<br>
<br>
```

2. Add the following code for an address text area:

```
<label>Address:</label>
<textarea cols="80" rows="5"></textarea>
<br>
<br>
```

3. Save the file.

3.8. Add a Password Field

A password field masks the characters the user enters.

1. Add the following code for the password field:

```
<label>Password:</label>
<input type="password" id="pass" name="pass"/>
<br>
<br>
```

2. Save the file.

3.9. Add the Submit Button

When the user clicks the submit button, the browser will open the page listed in the form's **action** attribute. For this lab, the form will display a local thank-you page.

1. Add the following code before the closing **form** tag:

```
<input type="submit" value="Submit"/>
```

2. Your completed **RegistrationForm.html** file should look like this:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8"/>
    <title>Registration Form</title>
</head>
<body>
    <h1>Registration Form</h1>
    <form action="ThankYou.html">
        <label>Firstname:</label>
        <input type="text" name="firstname" size="15"/> <br> <br>
        <label>Lastname:</label>
        <input type="text" name="lastname" size="15"/> <br> <br>
        <label>Course:</label>
        <select>
            <option value="Course">Course</option>
            <option value="BCA">BCA</option>
            <option value="BBA">BBA</option>
            <option value="B.Tech">B.Tech</option>
            <option value="MBA">MBA</option>
            <option value="MCA">MCA</option>
            <option value="M.Tech">M.Tech</option>
        </select>
        <br>
        <br>
        <label>Gender:</label><br>
        <input type="radio" name="gender"/> Male <br>
        <input type="radio" name="gender"/> Female <br>
        <input type="radio" name="gender"/> Other
        <br>
        <br>
        <label>Hobbies:</label>
        <br>
        <input type="checkbox" name="Programming"/> Programming <br>
        <input type="checkbox" name="Cricket"/> Cricket <br>
```

```
        <input type="checkbox" name="Football"/> Football <br>
        <input type="checkbox" name="ReadingNovel"/> Reading Novel
        <br>
        <br>
        <label>Phone:</label>
        <input type="text" name="phone" size="10"/>
        <br>
        <br>
        <label>Address:</label>
        <textarea cols="80" rows="5"></textarea>
        <br>
        <br>
        <label>Password:</label>
        <input type="password" id="pass" name="pass"/>
        <br>
        <br>
        <input type="submit" value="Submit"/>
    </form>
</body>
</html>
```

3. Save the file.

3.10. Create the Thank-You Page

The form uses the following action attribute:

```
<form action="ThankYou.html">
```

When the submit button is selected, the browser opens **ThankYou.html**. Create that file now.

1. In the Explorer panel, create a new file named **ThankYou.html**.

2. Enter the following code:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8"/>
    <title>Thank You Page</title>
</head>
<body>
```

```
    Thank you! Your form has been submitted.
</body>
</html>
```

3. Save the file.

3.11. Test with NGINX

1. Open a terminal in Visual Studio Code by selecting **Terminal** > **New Terminal**.
2. Copy both files to the NGINX document root:

```
sudo cp ~/html-labs/RegistrationForm.html /usr/share/nginx/html/
RegistrationForm.html
sudo cp ~/html-labs/ThankYou.html /usr/share/nginx/html/ThankYou.html
```

3. If prompted for your password, enter **labadmin**.
4. Verify that the files were copied successfully:

```
cat /usr/share/nginx/html/RegistrationForm.html
cat /usr/share/nginx/html/ThankYou.html
```

5. Open a browser.
6. Browse to the following URL:

   http://localhost/RegistrationForm.html

7. Enter information in the fields and select **Submit**.
8. Confirm that the browser opens **ThankYou.html** and displays the message **Thank you! Your form has been submitted.**

3.12. Review

In this lab, you created a basic HTML form with text fields, a drop-down list, radio buttons, checkboxes, a text area, a password field, and a submit button. You also created a thank-you page and tested both files through NGINX.

Improve Your Forms Using HTML5

LAB 4

This lab continues from the previous three labs:

- Learn the Basics of HTML Using Visual Studio Code
- Create a Basic HTML5 Layout
- Create a Registration Form

If you have not completed those labs yet, do that first before continuing.

In this lab, you will improve a form by using HTML5 input types and form attributes. You will work with the **email**, **date**, **number**, and **range** input types, and you will use the **placeholder**, **autofocus**, **pattern**, and **required** attributes.

The **placeholder** attribute displays hint text inside an empty field.

The **autofocus** attribute places the cursor in a field when the page first loads.

The **pattern** attribute uses a regular expression to limit which values are accepted.

The **required** attribute marks a field as mandatory.

4.1. Create the Employee Registration Form

In this part, you will create a new HTML file based on the template from the earlier lab.

1. If not already open, start Visual Studio Code.
2. Select **File** > **Open Folder**.
3. Open the **html-labs** folder you created in the previous labs.
4. If prompted, select **Yes, trust the authors**.
5. Create a new file named **registerEmployee.html**.
6. Use the structure from **html5Template.html** as a guide, then enter the following starter code:

```
<!DOCTYPE html>
<html>
<head>
        <meta charset="utf-8"/>
        <title>Register an Employee Form</title>
</head>
<body>
        <h1>Employee Registration Form</h1>

</body>
</html>
```

7. Save the file.

4.2. Add the Form Structure

1. Add the following form element below the heading:

```
<form action="ThankYou.html">

</form>
```

2. Add a full name field inside the form:

```
<label for="fullName">Full Name:</label><br>
<input type="text" name="fullName" id="fullName" placeholder="Enter your full
name" autofocus required> *<br>
```

Notice the following:

- The **placeholder** text helps the user understand what to enter.
- The **autofocus** attribute places the cursor in the full name field when the page loads.
- The **required** attribute means the field must be completed before the form can be submitted.

    1. Add an email field:

```
<label for="email">E-mail:</label><br>
<input type="email" name="email" id="email" placeholder="Enter a valid e-
mail" required> *<br>
```

Notice the following:

- The **email** input type helps the browser validate the value as an email address.
- The **required** attribute can also be used with this input type.

1. Add a date of birth field:

```
<label for="dateOfBirth">Date of Birth:</label><br>
<input type="date" name="dateOfBirth" id="dateOfBirth" required> *<br>
```

2. Add an age field:

```
<label for="age">Age:</label><br>
<input type="number" name="age" id="age" size="6" min="18" max="120"
step="1" value="21"><br>
```

Notice the following:

- The **number** input type allows numeric input.
- The **min** and **max** attributes define the allowed range.
- The **step** attribute controls the increment.
- The **value** attribute sets the default value.

1. Add a range field for years of experience:

```
<label for="rating">Years of Experience:</label><br>
<input type="range" name="rating" id="rating" min="0" max="100" step="1" val
ue="0"><br>
```

2. Add a phone field that uses a pattern:

```
<label for="phone">Phone:</label><br>
<input type="text" name="phone" id="phone" pattern="[0-9]{3}-[0-9]{3}-[0-9]
{4}" placeholder="XXX-XXX-XXXX" required> *<br>
```

Notice the following:

- There is no dedicated HTML5 phone input in this lab, so a text field is used.
- The **pattern** attribute restricts the format to **XXX-XXX-XXXX**.
- The **placeholder** attribute shows the expected format.

1. Add a submit button and a note for required fields:

```
<input type="submit" value="Register">

<p>Fields marked * are required.</p>
```

2. Your completed **registerEmployee.html** file should look like this:

```
<!DOCTYPE html>
<html>
<head>
        <meta charset="utf-8"/>
        <title>Register an Employee Form</title>
</head>
<body>
        <h1>Employee Registration Form</h1>
        <form action="ThankYou.html">
                <label for="fullName">Full Name:</label><br>
                <input type="text" name="fullName" id="fullName"
placeholder="Enter your full name" autofocus required> *<br>
                <label for="email">E-mail:</label><br>
                <input type="email" name="email" id="email" placeholder="Ent
er a valid e-mail" required> *<br>
                <label for="dateOfBirth">Date of Birth:</label><br>
                <input type="date" name="dateOfBirth" id="dateOfBirth" requi
red> *<br>
                <label for="age">Age:</label><br>
                <input type="number" name="age" id="age" size="6" min="18" m
ax="120" step="1" value="21"><br>
                <label for="rating">Years of Experience:</label><br>
                <input type="range" name="rating" id="rating" min="0" max="10
0" step="1" value="0"><br>
                <label for="phone">Phone:</label><br>
                <input type="text" name="phone" id="phone" pattern="[0-9]{3}-
[0-9]{3}-[0-9]{4}" placeholder="XXX-XXX-XXXX" required> *<br>
                <input type="submit" value="Register">

                <p>Fields marked * are required.</p>
        </form>
</body>
</html>
```

3. Save the file.

4.3. Reuse the Thank-You Page

This form uses the following action attribute:

```
<form action="ThankYou.html">
```

The **ThankYou.html** file was created in the previous lab. If it does not already exist in your **html-labs** folder, create it now with the following content:

```
<!DOCTYPE html>
<html>
<head>
        <meta charset="utf-8"/>
        <title>Thank You Page</title>
</head>
<body>
        Thank you! Your form has been submitted.
</body>
</html>
```

4.4. Test with NGINX

1. Open a terminal in Visual Studio Code by selecting **Terminal** > **New Terminal**.
2. Copy the updated files to the NGINX document root:

   ```
   sudo cp ~/html-labs/registerEmployee.html /usr/share/nginx/html/
   registerEmployee.html
   sudo cp ~/html-labs/ThankYou.html /usr/share/nginx/html/ThankYou.html
   ```

3. If prompted for your password, enter **labadmin**.
4. Verify that the files were copied successfully:

   ```
   cat /usr/share/nginx/html/registerEmployee.html
   cat /usr/share/nginx/html/ThankYou.html
   ```

5. Open a browser.
6. Browse to the following URL:

   http://localhost/registerEmployee.html

7. Confirm the following:
    - The **Full Name** field has focus when the page loads.
    - Placeholder text appears in the **Full Name**, **E-mail**, and **Phone** fields.
    - The **Age** field uses a numeric control.
    - The **Years of Experience** field uses a slider.
8. Test a valid submission with values such as the following:

    Full Name: Mike Harris

    E-mail: mharris@yahoo.com

    Date of Birth: 2000-01-08

    Age: 22

    Years of Experience: 0

    Phone: 215-314-2300
9. Select **Register**.
10. Confirm that the browser opens **ThankYou.html** and displays the thank-you message.
11. Select the browser Back button.
12. Clear the **Full Name** field and try to submit the form again.
13. Confirm that the browser prevents submission because the field is required.
14. Enter a name, then change the email value to **mharris**.
15. Try to submit the form again.
16. Confirm that the browser prevents submission because the email format is invalid.

4.5. Review

In this lab, you learned how to:

- Use the HTML5 input types **email**, **date**, **number**, and **range**
- Use the **placeholder**, **autofocus**, **pattern**, and **required** attributes
- Test the updated form through NGINX

HTML Video and Audio

LAB 5

As you go through the material for the HTML Video and Audio chapter, you can use the .html files included in the audio-video-youtube-examples folder to run all of the example code.

Note: If you are using the VM provided, unfortunately there will not be any audio. Feel free to run the examples in your own environment to hear the audio and to explore the example code.

Be sure to check the src= attribute in the source element of each html file to make sure the path is correct.

5.1. Example: Build a Simple Media Page

If you want one file that demonstrates audio, video, and an embedded YouTube clip, create a new file named **media-example.html**.

When working on Ubuntu with Visual Studio Code, use relative file paths so the example works regardless of where the folder is stored.

Add the following code:

```
<!DOCTYPE html>
<html>
<head>
        <meta charset="utf-8"/>
        <title>Media Example</title>
</head>
<body>
        <h1>Media Example</h1>

        <h2>Audio</h2>
        <audio controls>
                <source src="birds.mp3" type="audio/mpeg">
                Your browser does not support the audio element.
        </audio>

        <h2>Video</h2>
        <video width="320" height="240" controls>
                <source src="movie.mp4" type="video/mp4">
                Your browser does not support the video tag.
```

```
        </video>

        <h2>YouTube</h2>
        <iframe width="420" height="345" src="https://www.youtube.com/embed/
Uef5e1KzXRM" title="YouTube video player">
        </iframe>
</body>
</html>
```

Save the file, then open it in your browser. The page should display an audio player, a video player, and an embedded YouTube video on a single page.